School of Electronic Engineering
and Computer Science

Final Report

Programme of study:
Computer Science Masters with Foundation

Project Title:

Improving the Boundary Attack Method of Adversarial Image production to defeat Object Classification Systems

Supervisor:
Dr Pengwei Hao

Student Name:
Nicholas Franklin

Final Year
Undergraduate Project 2019/20

Queen Mary
University of London

Date: 11/05/2020

Abstract

Object classification systems are increasingly common as critical components in automation systems. These systems are increasingly acting in uncontrolled environments. Black-box methods of generating adversarial images are a threat to these systems, as they do not require knowledge of the internal structure of targeted system. As such they are more of a threat to commercial systems. Therefore, we need to find ways to make black-box adversarial methods more effective.

The Boundary Attack algorithm is the black-box algorithm I chose to improve, in order to better defeat object classification systems. This was with the aim to push improvements in object classification system robustness. In this project Boundary Attack was reimplemented in MATLAB, along with additional attempts at improving the algorithm, to find methods to increase the speed of the algorithm without unduly effecting the results.

The implementation varied in success with the reimplementation of a studied alternative to Boundary Attack failing outright. However, the novel Adaptive Boundary Attack method showed promise and requires further investigations.

Contents

Chapter 1: **Introduction**

1.1 Background

Automation has become common in highly controlled environments, such as, in factories. Unexpected events are unlikely and do not need to be accounted for in the system design. In addition to the significantly reduced risk that comes with not operating around people.

Automation in uncontrolled environments is becoming increasingly common, and for it to be safe, we require that systems are robust and can function in all plausible scenarios. The most common example being automated vehicles.

Image classification is a crucial component of the computer vision systems that are part of an automated system. However, these systems can be fooled or fail. If a computer vision system misinterprets an image, automation systems can take dangerous decisions.

Therefore, we need to push the robustness of these systems.

1.2 Problem Statement

The Boundary Attack (BA) algorithm produces adversarial images to cause image classification neural networks to misclassify images. It is effective against system hardened to other methods as discussed in the literature review. However, it is slow in producing adversarial images.

The BA algorithm needs to be improved further as it is a threat to image classification systems. Image classification systems need to be made more robust to attacks so that we can rely on them more in uncontrol situations with the increased use of automation. Hence the need to improve the Boundary Attack method to best push image classification neural network robustness to attack.

1.3 Aim

To develop improvements to the Boundary Attack algorithm to better defeat image classification neural networks. Thereby pushing image classification neural networks research to increase robustness to attack by malicious actors. To maintain high-quality images that are not easily identifiable as altered images.

1.4 Objectives

- Implement the Boundary Attack algorithm in MATLAB.
- Implement an improved proposal distribution for the Boundary Attack algorithm.
- Implement a heuristic that improves the initial image used in Boundary Attack.
- Investigate improving hyperparameter adjustment.
- Implemented an altered stopping function for Boundary Attack.

- Formally test the improvements made to boundary attack versus the implementation of the original algorithm.

1.5 Research Questions

Can the Boundary Attack algorithm be improved so that it is more efficient and can be better used to cause image classification neural networks to misclassify images?

Chapter 2: **Literature Review**

2.1 Concept Definitions

2.1.1 Definition of an adversarial image

'Adversarial examples are inputs to machine learning models that an attacker has intentionally designed to cause the model to make a mistake'. (Goodfellow et al., 2019) Therefore, an adversarial image is an image that an attacker has intentionally designed so that a model makes a mistake. In the case of classification systems, the mistake is outputting the wrong class for an image.

2.1.2 What are adversarial image perturbations?

Adversarial image perturbations are intentional changes in pixel intensity within an image to create a worst-case image for a classification system to classify correctly. An extension to this is minimising the difference between the adversarial image and an original non-adversarial image so that a system will misclassify it, but the original label of the image is apparent to a person.

2.1.3 Targeted Misclassification

Targeted misclassification is an extension to adversarial image creations. The image that is created is classified as a specific target class rather than the class that it appears to be to a person. (Brendel, Rauber and Bethge, 2018)

2.1.4 Black-box methods vs White-box methods of generating adversarial images.

There are two categories of adversarial image generation algorithms to describe how the information from or about the target system is used. In addition to what information is required to create adversarial images that will cause a targeted system to misclassify an image. These are black-box and white-box methods.

In a black-box algorithm, the only information that a system has access to is the output of a system and what is input into the system. In the case of an object classification system, this will typically be the output class that the system assigned to an input image. This output can vary greatly from a detailed list of confidence scores for each class to a one label decision. These different outputs can inform different methods.

In a white-box algorithm, the developer of the system has access to information about how the targeted system functions internally, beyond just the inputs and outputs. This information could be the internal structure of layers in a neural network, the pre-processing performed on an image before it is assessed, by a neural network, or the dataset that the system was trained on. (Brendel, Rauber and Bethge, 2018)

2.2 The Categories of Adversarial Image Generation

2.2.1 Gradient-based Attacks

Gradient-based attacks are white-box methods that use the gradient of the loss function to determine adversarial perturbations. The Fast Gradient Sign Method (FGSM) is an example of a gradient-based attack. In this attack, the gradient of the loss function for an input image at each pixel is used to determine if a pixel is to be increased or decreased in value by the parameter ϵ. The pixel value is increased if the loss gradient is positive and decreased if negative so that the magnitude of the loss (the gradient of the error) is increased. Therefore, resulting in a misclassification as the error increases.

This formula is used to define the perturbations in the image.

$$\boldsymbol{\eta} = \epsilon \text{sign}\left(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y)\right).$$

The perturbation η is for a given pixel is scaled by ϵ multiplied by the sign of the gradient. $\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y)$ is the gradient of the loss function with respect to the input image x, y being the true label of the input image and θ being the weights of the target neural network. The gradient is calculated using the backpropagation algorithm. (Goodfellow, Shlens and Szegedy, 2015)

2.2.2 Transfer-based Attacks

Transfer-based attacks are a black-box attack that aims to produce a neural network that simulates the operation and output of another target neural network. Transfer-based attacks aim to build up a ‘substitute’ dataset for the targeted model using a synthetic dataset created by repeated querying of the targeted system. The input images are then labelled with the outputs of the targeted system when classifying them, and this new synthetic dataset is then used to create adversarial images. A neural network is trained using the synthetic labelled dataset. This model is then defeated using a selected white-box method. As the internal structure is known as are parameters of the substitute model, a white-box method can be used at this stage. Finally, the images created that are adversarial against the substitute model are highly likely to be adversarial against the targeted model. The adversarial images are transferable between the two systems. (Papernot et al., 2017)

The Papernot Transfer Attack (PTA), as described above, creates a substitution set. This set is then used to train a network, and adversarial images are then made against this network using the FGSM or the Jacobian-based Saliency Map Attack which are both Gradient-based attacks.

PTA was found to cause misclassification rates of greater than 62% against systems not trained to protect against it, with images that were still human recognisable.

2.2.3 Decision-based Attacks

Decision-base attacks are black-box attacks that only required the output class of the classification neural network, in the form top-1 class label.

They differ from transfer attacks as they create adversarial images directly aimed at the target system, rather than creating a substitute dataset and defeating a secondary neural network trained on this data. This requires a transfer of the adversarial images that defence methods can make ineffective. A

decision attacks only required the image that you wish to make adversarial to function as an input and the ability to query the output of a neural network given a specific image. BA is an example of a decision-based attack.

2.2.3.1 How Boundary Attack functions

BA operates by feeding images into a classification system and evaluating the output against the 'adversarial criterion' to determine if an image in an iteration of the algorithm is correctly adversarial.

It only requires the output label of a system when given a specific input image. However, the adversarial criterion can be adapted to use additional information if more is output. The decision of the system is used in the adversarial criterion to determine if the image is correctly adversarial at each stage of the algorithm, so more information simply makes a more informed criterion.

The adversarial criterion can be as simple as "Does the targeted system assign the wrong class to an image?" to "Does the correct class of the image appear outside the top 5 possible classifications of the image?" if the network provides ranked class predictions.

BA is first initialised with an already adversarial image. In the original proposal of this method, a simple uniform distribution 0,255 is sampled at each pixel to build the initial image. This initial image is extremely unlikely to be classified as the target image you wish to process, therefore fits the adversarial criterion.

BA then iterates through images adding new perturbations and moving the adversarial image closer to the original image that we wish the perturbed image to look like.

At each iteration, if the image is found to be correctly adversarial, then a new layer of perturbations are added to the image using the proposal distribution.

The proposal distribution defines constraints that the perturbations need to follow before they are added to the image. These constraints are:

a. *The newly perturbed image must be within the input domain. (In the case of RGB that is in the range 0 – 255).*
b. *The perturbations need a total absolute length less than the hyper parameter* δ.
c. *The perturbations must reduce the difference between the perturbed image and the original image.*

These are simplified into a heuristic.

First, a sample is taken from an independent and identically distributed Gaussian distribution with a mean of 0 and standard deviation of 1.
Then the sample is rescaled, and values clipped so that the image obeys constraint a and b.

Next, the rescaled sample is added to the previous iteration (or the initial image on the first iteration) and then it is compared to the original image.

The difference between the original image and the perturbed image with the new perturbation added should be equal to the difference between the original image and the perturbed image, without the new perturbation added.

This is the addition of orthogonal perturbations as they change the perturbations but do not bring the image closer to the original. The failure rate of orthogonal perturbations in staying adversarial is used to adjust the size of δ (the total perturbation length).

Finally, a small movement towards the original image is taken such that a and c hold so that the new image is closer to the original. The rate of failure of the in step is used to tune the hyperparameter ϵ. ϵ being the lengths of the step towards the original image.

Hyperparameter adjustment is performed dynamically to attempt to match the boundary between adversarial and non-adversarial images. The method used was inspired by the Trust Region method. The orthogonal step δ is made to test if the local region can be assumed to be approximately linear. With this assumption, then you would expect that 50% of the orthogonal steps are adversarial. If greater than 50% are adversarial, the size of the steps is increased. If it is smaller, the size of the steps is decreased. Similarly, the in step ϵ is used to move closer to the original image. The rate of failure on the in step is used to adjust the size of the steps, similarly, increasing if the rate of failure is low and decreasing it if the rate of failure is too high.

Hyperparameter adjustment also gives BA its stopping condition. That is when the in step ϵ converges to zero.

2.3 Defending against the Studied Methods

2.3.1 Defensive Distillation to Defeat Gradient-based Attacks

Distillation is a training operation originally designed to train simpler neural networks using knowledge from a larger neural network. This was intended to allow deep learning functionality on less powerful devices like smartphones.

Distillation functions by first training a classification neural network on the training set. The resulting distilled knowledge is a series of probability vectors for each image in the training set. In standard distillation training, these probability vectors would be used to replace the true labels of the training dataset. This dataset would then be used to train simpler networks on power-limited devices.

Defensive Distillation is an alteration of Distillation to defend against adversarial images. In Defensive Distillation, the probability vectors from the initial neural network are used to augment the training dataset used to train a new neural network with the same architecture as the original. Therefore, there is additional information contained in the probability vector label of what the correctly classified image should be similar to in addition to what it should be classified as. This creates a model that is less tightly fitted to the data, therefore, is more general and much less vulnerable to some adversarial images. (Papernot et al., 2016)

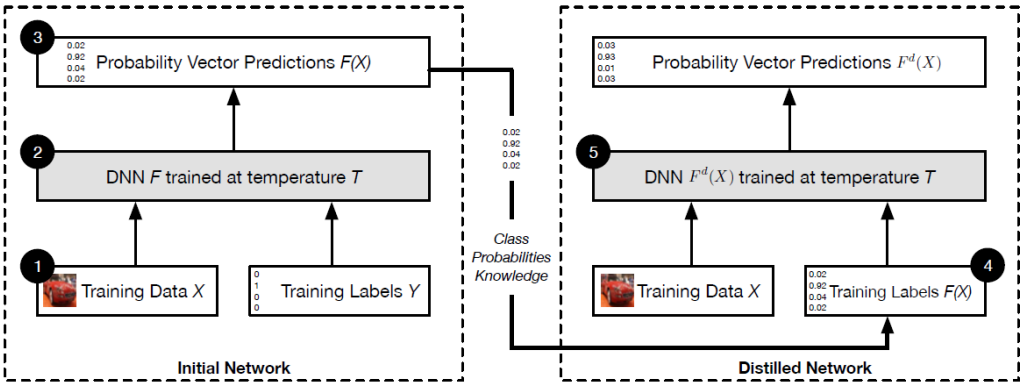


Figure 1 (Papernot et al., 2016)

2.3.2 Ensemble Adversarial Training to defeat Transfer-based Attacks

PTA has been defeated by the use of Ensemble Adversarial Training (EAT). In EAT, a neural network that is to be protected from black-box attacks, are trained on perturbed images. These perturbed images are generated by attacking a pre-trained model on a non-augmented data set. These adversarial images are then used to augment the training set of the new network taking advantage of the transferability of adversarial images between models to produce a new network that is more robust to adversarial images being generated against it.

Transfer-based attacks will find it hard to create the adversarial image from the substitution set as the substitution set is not going to contain the adversarial images that this model was training on. Therefore, it will create adversarial images similar to the adversarial images that the protected system can already classify correctly. (Tramer et al., 2018)

2.4 Faults and Benefits of Boundary Attack

2.4.1 Faults with Boundary attack

The major flaw with BA is the number of iterations needed to conclude the algorithm. This makes it costly compared to other algorithms that generate adversarial images. Therefore, the main area for improvement is in execution speed. As seen in *figure* 5, over 200,000 calls to the model system are made by the time ϵ converges to zero. The image is imperceptibly perturbed at only 8272 calls. Nevertheless, this is still slow compared to other methods. For example, DeepFool (a gradient-based attack) only needs 7 calls to the model system and 37 gradient calculations. (Brendel, Rauber and Bethge, 2018)

2.4.2 Benefits to Boundary attack

It is very easy to change its adversarial criterion. Allowing for a very flexible system, for example, you can create a targeted misclassification by swapping out the initial image and defining the adversarial criterion to “Was the image classified the same as the initial image?” Resulting in an image that is classified as a specified different class.

Furthermore, it is successful against systems that are hardened against adversarial attacks while achieving results with smaller errors to the original image than other methods. In a comparison between BA and FGSM against distilled and non-distilled networks, it is successful in both cases where FGSM failed against the distilled network. Furthermore, FGSM achieved worse image quality against a standard network. (Brendel, Rauber and Bethge, 2018)

		MNIST		CIFAR	
	Attack Type	standard	distilled	standard	distilled
FGSM	gradient-based	4.2e-02	fails	2.5e-05	fails
Boundary (ours)	decision-based	3.6e-03	4.2e-03	5.6e-06	1.3e-05

Figure 2 (Brendel, Rauber and Bethge, 2018. MNIST and CIFAR are the dataset the network was trained on.

2.5 Universal Adversarial Perturbations

Universal adversarial perturbations is the concept that there exists for a given neural network a series of small perturbations that can be applied to any image that the system would normally classify correctly, with the result that the image is misclassified. These are proved to exist in (Moosavi-Dezfooli et al., 2017).

This research further shows how easily disrupted neural networks are. The robustness of neural networks is a crucial area that needs to be improved so that they are not susceptible to attack when they are used in critical systems.

In addition, the universal perturbations generated show a high level of universality between different systems trained on the same dataset. Allowing the defeating of an unknown system with precalculated perturbations.

2.6 Automation in an Uncontrolled Setting

Classification systems must be made as robust as possible because they are a core part of automation in the form of computer vision systems. As automation is increasingly being used in an uncontrolled environment, we need it to be robust against attack.

In the past, computer vision systems were mostly used in controlled settings where the content that the system would be processing was always known. Therefore, high levels of robustness were not needed. For example, you do not need a particularly robust system to identify unripe fruit on a belt as there is little risk if the system fails and little room for adversarial attack and unexpected events.

With the increasing shift to automation in uncontrolled environments, this changes. In an uncontrolled environment, you do not have certainty in what the system will come up against. The best example of this is automated vehicles which need to navigate roads that contain other actors that can act erratically.

Automated vehicles take actions based on their perception of the world around them. This includes signs, road markings and other vehicles. If an automated vehicle fails to classify a critical object correctly, it might perform unsafe and unexpected behaviours leading to an accident.

Adversarial perturbations could be used to cause unexpected behaviour, for example, a stop sign might be modified by a malicious actor so that it appears to say stop to a person, yet interpreted as a minimum speed sign by a computer vision system. These attacks would be hard for people to identify as the images produced can be made to be indistinguishable from the original through targeted misclassification. This would pose a significate risk to automated vehicles that are targeted by it.

There has already been a failure in a partially automated car that has resulted in a death because a computer vision system failed to identify a critical object correctly. A Tesla failed to identify a white trucks' side as a truck was because it was presented against a bright sky. So the car did not correctly apply the brakes. This resulted in the car crashing into the truck and the death of the occupant of the car. This shows the risk of misclassification in automation and a need to improve robustness to protect against edge cases and adversarial attack. (Tesla.com, 2019)

This year, McAfee, using white-box methods, caused a Tesla to accelerate by 50 miles an hour while the car was on automated speed control. They made a simple modification to a speed sign to induce an acceleration that their white-box method informed. Confirming that adversarial attacks are a serious risk for automated vehicles. In addition, the authors discuss the applicability of this research to black-box attacks and the transferability of adversarial images. (Povolny and Trivedi, 2020)

Figure 3 The Adversarial Speed Sign

2.7 Critical Analysis of the Literature

I chose to focus my literature critique on the two papers that were most influential on the project.

2.7.1 A Brief Review of Brendel, Rauber and Bethge, 2018, 'Decision-Based Adversarial Attacks: Reliable Attacks Against Black-Box Machine Learning Models'

The abstract well summarises the class definitions they present in the introduction as well as an overview of how BA functions and an indication of the conclusions and results of BA.

The BA explanation is detailed with links to the implementation that contain further details to allow for replications and excellent diagrams to illustrate concepts. In addition to good logical flow between sections.

Finally, the conclusion reaffirms the links between Boundary Attack and the rest of the paper while discussing possible future areas of study.

2.7.2 A Brief Review of Guo, Frank and Weinberger, 2019, 'Low Frequency Adversarial Perturbation'

Guo, Frank and Weinberger do not clearly show their failure rate in achieving their 0.001 mean squared error (MSE) metric. This can be seen in Fig 6 where they compare progress in average MSE reduction but do not state what proportion failed. This statistic would be useful for comparison to this project and would be useful in considering reduction in said failure rate.

The title is too short giving little to no insight into the purpose of the paper. The purpose of the paper is well shown in the text. It aims to show that low-frequency space is valuable in the search for adversarial perturbations.

The testing methodology is clearly explained with a detailed breakdown of each test being performed and an explanation of the results.

Finally, the conclusion much like the abstract is short but is a good overview of the content of the paper with future applications of this research discussed.

2.7.3 Meta Criticism

A large amount of the direct research into adversarial image generation has been performed by a couple of similar groups of researchers, with Ian Goodfellow, or Nicolas Papernot on a high proportion of teams. This bias in the composition of research groups could reduce the quality of the research.

Further to that, I found little to no contradicting or conflicting research in this space. I believe this is because the research area is rather small with direct adversarial image generation papers citing a lot of the same work with most papers referencing each other.

2.8 Why I chose this research area?

I wanted to implement and improve on a complete algorithm. I have not had the opportunity to study, implement and improve on a proposed algorithm in a formal manner, and I was interested in using the project to explore this. I chose BA because I found its manner of operation interesting, it was an effective black-box attack, and the authors state that 'there are many ways in which Boundary Attack can be made even more effective.'

I was interested in the concept of preventing other parties from extracting data from my images. This was part of the original motivation but not a factor anymore, because in my research I have seen that this would probably not be useful for very long because of the speed of progression in the field.

Why Study Boundary Attack?

I want to improve BA further because it is a black-box method and black-box methods are the most realistic threat to computer visions systems as they rely only on the output of the system which is being targeted. The dataset, hidden layer structures and parameters used are not typically going to be available when targeting commercial applications like automated vehicles. Therefore, we need to develop black-box methods further so that neural networks and by extension, computer vision systems can be hardened against these attacks. Making them safer and allows for further increases in automation usage. In addition, Boundary Attack was proposed recently, 2018, so there is likely room for improvement as little research has aimed to improve it. The only paper I have found directly proposing an improvement to BA being (Guo, Frank and Weinberger, 2019).

Chapter 3: **Design and Development**

3.1 Requirements of the Design

The requirements of the system architecture were informed by the methods of improvement to be explored and the literature.

The literature shows several distinct components to BA, which were the focus of the project. The improvements discussed in this chapter show that the main requirement in the project was flexibility as the suggested methods were all replacements of components should be made modular.

In addition, there was no expectation of altering the core operation of BA beyond parameters. This requirement was used to create a class diagram that shows how components are related. Shown in appendix D.

In addition, I created a sequence diagram to show the path of execution to generate an adversarial image and the points at which the core system interacted with the interfaces representing the various components.

The Adaptive Boundary Attack class and sequence diagram show the alteration to the design that the Adaptive method required because it needed changes to the Boundary Attack core to function.

The input and outputs requirements were defined by the inputs and outputs of the original proposal. Therefore, the system must be able to query a neural network with a specific image and receive a label as a prediction but no more. In addition, the system must take a labelled image as an input and output an adversarial image derived from the input image.

3.1.1 Improving the Proposal Distribution

3.1.1.1 Low Frequency Proposal

An improved proposal distribution has already been identified as a point of improvement in Boundary Attack. (Guo, Frank and Weinberger, 2019). This paper discusses the importance to focus adversarial image perturbation algorithms on lower frequencies. A grey level slicing of an image shows that the least information in the image is in higher frequencies of the image and the higher frequency levels showing little to no details. *figure 3* This indicated that adversarial perturbation would be more likely to function at lower frequencies.

This paper showed that more adversarial examples exist in the lower frequencies and that it is best to focus searches in this space. In addition, this has the benefit of avoiding defence methods that can defeat adversarial examples like blurring the input image. Blurs remove higher frequencies and adversarial images that only perturb higher frequencies can be defeated if a simple blur is performed before classification.

Furthermore, image compression often removes higher frequencies, for example, jpeg compression. Therefore, these perturbations pose a greater risk to more systems.

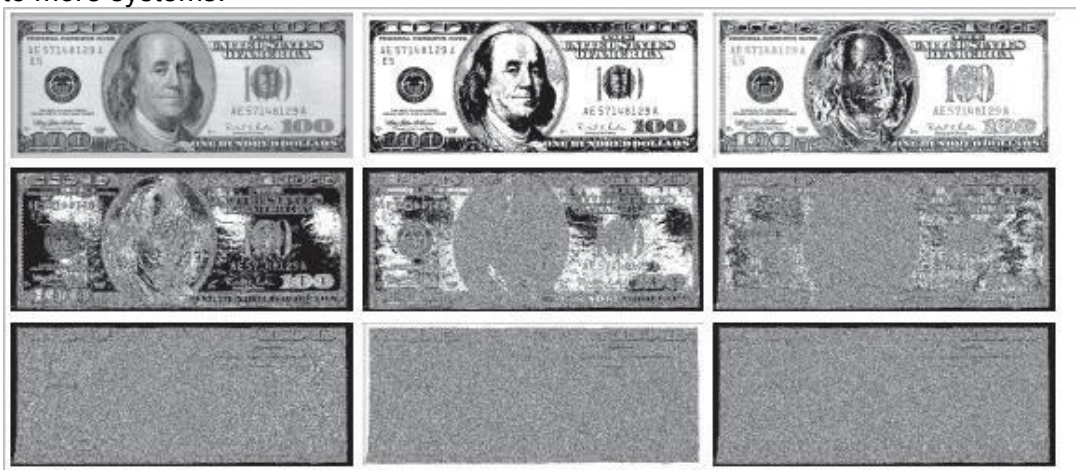


Figure 4 (Gonzalez and Woods, 2017)

Figure 5 (Brendel, Rauber and Bethge, 2018)

3.1.1.2 Adaptive Proposal

The original intension for the Adaptive Proposal was to be a proposal distribution that took advantage of the statistics in a similar manner to the Hyperparameter adjuster to alter how it internally function without alterations to the core of BA. Late in development, this changed resulting in the design not working as well for the version of the Adaptive Proposal as implemented.

3.1.2 Improving Initial Image

A heuristic to improve the original image could improve the speed of the algorithm. The initialisation at 0 calls is a uniform distribution between 0-255 as seen in the first image of *figure 4*. This is far from the original image that we want the adversarial image to be similar to. If a heuristic could be created that would generate an initial image closer to the original yet correctly adversarial then execution could be sped up.

3.1.3 Improving Stopping Function

Some of the output images of the BA algorithm are extremely high quality with little to no visual difference between processed images and the original. This high-quality level is not required in real-world adversarial attacks. A drop in the visual quality would probably not be noticeable to people under standard conditions while still, as per the function of BA, fool the targeted classification system. Execution times would be reduced. *figure 4* shows that the improvement in image quality between call 42213 and call 200667 is extremely small, with a MSE increase of only 4.9×10^{-6}.

3.1.4 Improving hyperparameter adjustment

Altering the hyperparameter tuning to speed up the algorithm is a possible area for improvement. The original implementation was inspired by the Trust-Region

method. Therefore, I reason that an optimised or alternative version of the Trust-Region method could improve the execution speed.

3.2 Development

3.2.1 Development Chronology

In the first stage of development, the functions required for Boundary Attack were created. This included the uniform image generation and the generation of low-frequency noise, as these did not require a deep understanding of BA.

Then a skeleton of the system was created following the initial class diagram. With the skeleton finished, core components were developed. This included the Trust Region inspired hyperparameter adjuster and the neural network querying component. In addition, the functions developed initially were incorporated into the skeleton component classes.

The Boundary Attack core class was programmed, followed by the Low-Frequency proposal. Then the various alternative initialisation methods were developed. Starting with the Informed Method. The Low Frequency Initialiser was developed as a test to compare to the standard Uniform Distribution Method, and it was a stage in creating the Low Frequency Informed Method.

Finally, the Adaptive Boundary Attack class was created due to the limitation posed by previous development.

3.3 Component Implementation

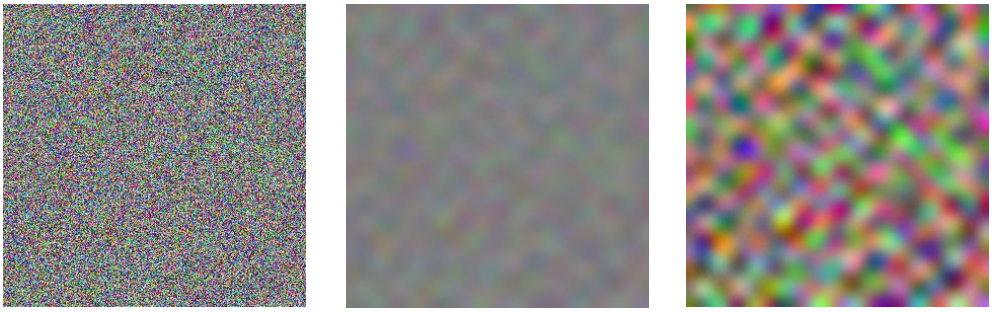
3.3.1 Proposal Distribution Noise Generation

Inverse Discrete Cosine Transform (IDCT) sampling is the primary method I used to generate low-frequency noise images. It was used in (Guo, Frank and Weinberger, 2019) for their implementation of the Low Frequency Proposal.

In this method, a Discrete Cosine Transform (DCT) is performed on each channel of an IID Gaussian image matrix. The transform creates a matrix describing the original image in terms of sums of frequencies in an image with the sum of low frequencies occupying the top left corner and high frequencies as you move further to the bottom right.

The Transform matrix then has all values outside of a range from the top left in both dimensions zeroed. The new matrices are then transformed using IDCT to give a three-channel image without the higher frequencies present.

The low frequencies are low magnitude so have to be rescaled to cover the whole 8-bit range. The range can then be scaled to fit its purpose. The Adaptive Proposalw



IID Gaussian Sample *IDCT Sample* *Rescaled IDCT Sample*

Figure 6 The stages to generating low frequency noise image using a reduction to 1/16 of the original frequencies present.

3.3.2 The Adaptive Proposal Implementation

The Adaptive Proposal (AP) is the alternative proposal distribution developed in this project. It aims to take advantage of successful iterations to try and get a further reduced error without having to re-query the target neural network as often.

The Adaptive Boundary Attack class was required due to limitations in the design, making the intended implementation impossible using the normal BA class. Changes were needed to the base BA class because alterations had to be made to when the neural network was queried to avoid querying an image known to be successful.

AP operates by monitoring if the previous in step or orthogonal image were successful. If both the in step image and the orthogonal image are successful, then the in step image is recreated using a greater step size. This step in is increased by multiplying by the repeat adjustment parameter to the power of the number of successive repeat steps. The same orthogonal image is used, as it is known adversarial. The new in step image is tested against the adversarial criterion, only a single query is required to the neural network as we know that the orthogonal image was already successful.

Furthermore, AP tries to take advantage of partly successful iterations. If the orthogonal image was successful, but the in step was not, a new in step is created using a shorter in step length created by dividing the current step length by the repeat adjustment parameter to the power of the number of successive repeat steps.

In addition, the number of reuses of the orthogonal image is capped to avoid loops of no progress after a successful orthogonal image. AP takes as a parameter another proposal distribution for orthogonal and instep image generation on non-repeat steps.

In testing a value of 1.5 was used for the repeat adjustment parameter with the backup proposal distribution being the standard gaussian proposal. The maximum repeat steps in succession was varied in testing.

IF NO. repeated orthogonal image **<=** Max NO. repeated orthogonal image

IF previous orthogonal image **AND** previous step in image

Increment the number of time the orthogonal images has been repeated.

Recalculate the in step image with the step length multiplied by the repeat adjustment parameter raised to the power of the current number of successive repeat steps

ELSE IF previous orthogonal image

Increment the number of time the orthogonal images has been repeated.

Recalculate the in step image with the step length divided by the repeat adjustment parameter raised to the power of the current number of repeat steps.

ELSE

Sample the backup proposal

ELSE

Sample the backup proposal

Return the orthogonal image, the in step image and if the orthogonal image was reused.

Adaptive Proposal Pseudo Code

3.3.3 Modified End Condition

I made several changes to the end condition to meet a set of restrictions.

An attempt will end:

- If progress is not being made in error reduction.
- If the error rate is imperceptibly low.
- If the iteration limited is reached.

I chose to end a run if no improvements have been made in the Mean Square Error of the adversarial image in 400 iterations of the algorithm. Due to the random nature of these processes, if set to low, it would cut off before the algorithm has reasonably converged. I selected 400 because it allowed for 6 full batches of hyperparameter adjustment. If no more progress was made in this time, I deemed no more meaningful progress would happen.

I chose to use a 5000 iterations limit to better distinguish between speed enhancements that changes I make produce. Most changes should not greatly change the final output image when run to an ϵ of zero. The most relevant changes to the speed of execution will be evident early, and I wanted to have more computing time to test more parameters of my improvements rather than comparing fewer runs to absolute completion.

I assumed that an error rate of 1 x 10^{-7} would be visually imperceptible to the original on all images so would be suitable to end here as no meaningful progress can be further made. This value would be best determined through user studies I intended to do but were cut due to time considerations.

3.3.4 Hyperparameter Updating

I was unable to identify a plausible alternative method for hyperparameter adjustment than the Trust Region inspired method that was suggested in the original proposal of BA.

This method tried to keep the success rate between the two boundaries. If the rate is too high progress can be made quicker. If it is too low, then the steps being made are too high. Hyperparameters are adjusted up or down by multiplying or dividing by the adjustment parameter. This algorithm works in batches with the success rate adjusting the hyperparameters for the next batch.

I made a mistake during the implementation of this method. This resulted in hyperparameters that continuously updated rather than by batch. This occasionally yielded better final errors in a shorter time. However, it was extremely unstable with massive growth or contraction of the parameters common. I chose to focus on the known working method first and was unable to explore methods to improve this possible alternative method. I had considered a rolling batch method as a hybrid that might see and improvement as it heads off a need to change more slowly, but that will be left for future work.

3.3.5 Initial Image Generation

Almost anything can be used as the initialising image in Boundary Attack. The image simply needs to be classified as something other than the true label of the original image. The lack of an attempt to increase execution speed drew me to this area.

I implemented the standard method (sampling from a uniform distribution at each pixel), a rescaling a matrix of IID gaussian samples and a low-frequency method using IDCT discussed previously. I do not expect to see any improvements from the latter over the standard implementation as this did not derive any information from the original image so would not reduce the error to the original.

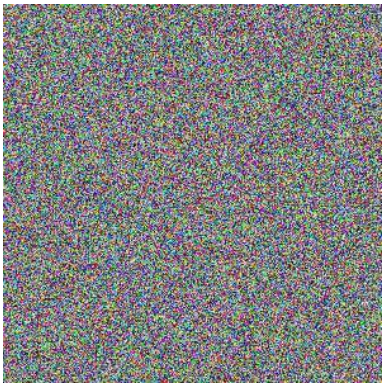
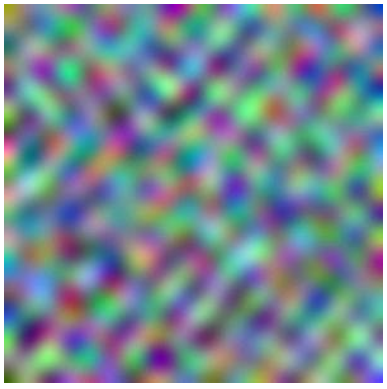


Figure 7 Scaled IID Gaussian Noise

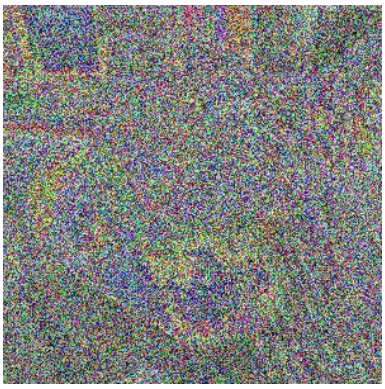


Low Frequency Noise 1/16

Following on from this I implemented a method which I took the original image and bit shifted it to remove some number of the most or least significant bits. In the now-empty bits, I added in noise as generated in the previous methods to further increase the chance of a misclassification while reducing error to the original.

I called this the Informed method. The Informed method had one integer parameter representing the number of grey levels replaced with noise, positive being the number of least significate bits removed and negative the number of most significant bits removed, referred to as Positive Informed (PI) method and Negative Informed method (NI).

Informed -1

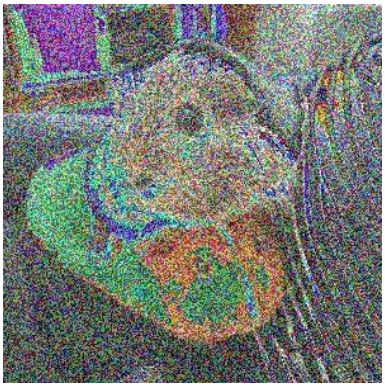


Informed -2

Informed 7

Informed 6

Low Frequency Informed 7



Low Frequency Informed -1

Figure 8 A comparison of Informed Initialisation methods.

3.4 Challenges in Development

3.4.1 Expected challenges

I was new to MATLAB and knew this would slow down development as I would make mistakes due to my lack of experience. This was excepted and accounted for when I decided to use MATLAB over Python. I made this decision to increased

development speed, as MATLAB allowed for easier inspection of the data and plotting which benefit this project greatly.

I expected implementing BA to be extremely difficult.

3.4.1.1 Unforeseen Challenges

3.4.1.1.1 Implementation Challenges

Programming a flexible system that I was able to test many different components with was a challenge I identified early on. I approached this with a class diagram to structure the system and help to lower coupling in the system.

This approach, for the most part, worked well with most core aspects of the algorithm being interchangeable with different components implementing the same functionality.

Hyperparameter adjustment became more coupled than expected under my original design. I left it as is because I had not been able to identify any viable Trust Region alternative at that time in the project, and I saw it as a challenging area to find improvement.

Generating low-frequency noise matrix was useful for some of my improvement attempts and the implementation of the Low Frequency Proposal from (Guo, Frank and Weinberger, 2019).

I investigated various methods to generate low frequency noise expecting it to be a straightforward process. Including scaling images sampled from various distributions, algorithmic noise and other transforms. However, I ended up using the approach proposed in paper (Guo, Frank and Weinberger, 2019) with alternative method becoming an extension as other areas had more promise than altered noise generation methods in getting an improvement.

3.4.1.1.2 Dataset Challenges

I assumed that all the images in the dataset would be standardised in some properties. Image dimensions were variable as expected, but I did not expect there to be any single-channel images. This was discovered when my program crashed over 700 images into a run. The solution was to simply duplicate the layers of single-channel images into three channels when found.

There were also problems with homonyms in the dataset labels and multiple label classes. The ImageNet dataset has 1000 classes with one to nine different labels. There were duplicate labels between some classes which could cause issues with an image being incorrectly classified as the homonym of its true label. I solved this by removing the duplicate label from the class that had the most labels. This presented the further problem of ‘crane’, the bird, and ‘crane’, the construction machinery. As both are single label classes. I opted to remove the class of crane, the bird because birds are common in the dataset.

3.4.1.1.3 Testing Challenges

I had to change the target neural network several times during the project. I originally intended to use NASNet-Large because it is the most recent pretrained neural network available in MATLAB. I was limited to neural networks that were trained on the ImageNet dataset as my testing required a large labelled dataset, and this was the dataset that I had access to.

I encountered my first issues when I started testing because the predicted run time for processing 1000 images was over six days. This was due to the lack of GPU acceleration for NASNet-LARGE in MATLAB. I swapped to Inception-v3 as the target network and then to GoogLeNet when the run times was still predicted over 24 hours even with these GPU accelerated networks.

I had a further unexpected challenge when I solved a performance problem extremely close to the deadline. The system produced better results but had to be fully retested with 16 days left to the project deadline. To maximise my remaining time, components parameters with the best chance of success were tested first, and further testing was informed on these results.

Chapter 4: **Results and Discussion**

4.1 Testing Methodology

4.1.1 Method of testing

In testing, various configurations of BA were run over 1000 test images. The test images represented one of each class of image contained in the ImageNet data set. Initially, each configuration only changed one component or component parameter compared to standard BA.

Raw statistical data was recorded for every original image within each configuration of BA. Output images at various iterations were saved.

Further data was derived from the raw data of the runs of BA to judge the performance of each of the configurations. The new data was evaluated to informed further testing using the most promising parameters.

Here, I found the Low-Frequency Proposal was performing badly with results similar to the standard, so testing stopped including it. As shown in figure 10

4.1.2 Methodology Justification

The intention was to perform two waves of testing. One on a larger volume of images at random and a second on a small set of images to a higher quality level. Due to time constraints caused by fixing a bug extremely late in the writing of the report, I had to retest and rewrite extensively.

Therefore, testing was amalgamated into a single test of over 1000 of the same images for each configuration. This reduces the strength of my conclusions as more testing would be preferable, but this was deemed the best compromise to finishing the project with useful conclusions.

4.1.3 Testing Metrics

These are the metrics I use – query count, iteration count, query count iteration ratio, initial error, run time, failure rate, and average final error.

Query count, iteration count and the ratio of the two are system agnostic metrics useful for comparison. The query count is simply the number of times the target neural network was called in a run. The iteration count is how many iterations did a run perform before stopping.

The ratio of query and iteration count is another useful metric because querying the neural network is a costly process and if the ratio is lower than less queries are being made over a run of BA leading to greater efficiency.

Run time can be a good intra-system metric of comparison in the same implementation tested on the same system. However, it is variable even on the same system so needs to be used carefully as not to produce misleading results.

For the standard implementation of BA, there should never be an image produced from the system that is non-adversarial. Modifications made to improve BA could lead to the introduction of a failure rate, which needs to be weighed against the benefits they provide.

Finally, average final error is useful when comparing how different methods fair in reducing the error between the adversarial image and the final image over a large number of tests.

4.2 Statistical Results Discussion

4.2.1 Comparison of Initialisation methods Image Initialisation

4.2.1.1 The Informed Methods

There was some success in the Informed methods. The Positive Informed method, removing the least significant bits, saw a massive reduction in the initial error. From an average initial MSE of 0.1577 for the standard initialisation down to 0.0482 for Informed .

However, as expected, this introduced a failure rate in initialisation. For Informed 7, this failure rate was low. 76 initialisations failed out of 777 images that passed the sanity check (can the network correctly classify the original image). As the number of grey levels removed from the original image was reduced, the failure rate increased dramatically at Informed 5 over 75% of initialisations failed.

This trend continues with the low frequency informed method with the low frequency informed performing slightly worse on failed initialisations. Probably due to its lower average initial error.

The images that the Informed method caused to fail were the images that were likely to have a poor final error. This is shown by an improvement in the average final error as the failure rate increases.

The negative values for the informed method performed badly. They saw no decrease in the initial error compared to the standard implementation.

Improvements in initial error are marginal in improving the overall speed due to the fast reducing in error in the early stages of the algorithm.

Test Category	Initialisation Failure	Average Final Errors	Average Initial Error
Informed 5	602	2.402E-03	0.01540
Informed 6	323	3.222E-03	0.01742
Informed 7	76	7.197E-03	0.04821
Informed Low 5	627	8.103E-04	0.01449
Informed Low 6	376	2.607E-03	0.01899
Informed Low 7	75	7.097E-03	0.03170
Low Frequency	1	9.937E-03	0.10169
Standard (Uniform Distribution Sampling)	1	1.117E-02	0.15765

Table 1 Initial Image Comparison

4.2.1.2 Low Frequency Initialisation

Low-frequency Initialisation was expected to work equally as well as the Uniform Distribution Initialiser, as no attempt had been made to reduce the initial error or speed-up conversion. However, it had a significantly lower than average initial error 0.1017.

I am not completely sure why this occurred. My most convincing suggestion is that the low-frequency images occupy more of the intensities in the middle of the 8-bit range with less extreme blacks and whites. As a result, they mirror the photos used in the dataset, which are of real objects so will not typically contain pure white and black.

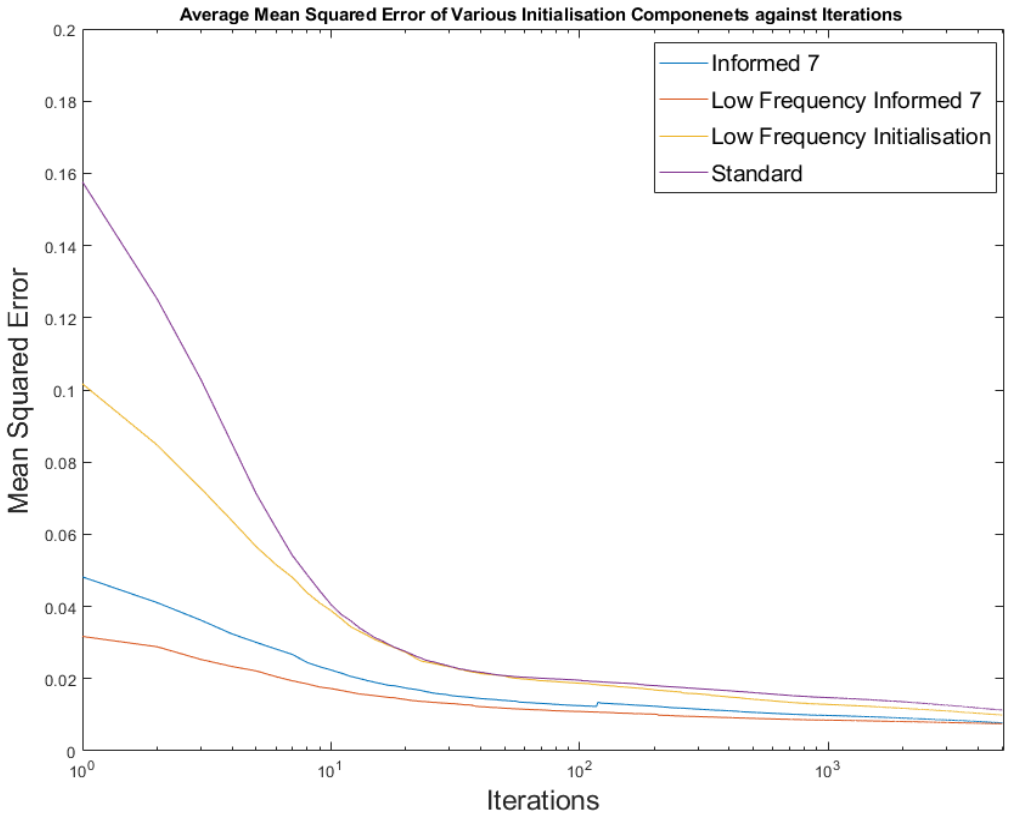


Figure 9

4.2.2 Proposal Distribution Comparison

4.2.2.1 The Low Frequency Proposal

The Low Frequency Proposal performed worse than expected. It showed little difference from the standard implementation, as shown in above. In addition to the same or higher average runtime. This is in contrasts to the expected improvement seen in (Guo, Frank and Weinberger, 2019). This indicates an implementation error in this method resulting in its lacklustre performance. An implementation error went unnoticed because of other cumulative bugs masking the fact that its performance was not different from the standard BA.

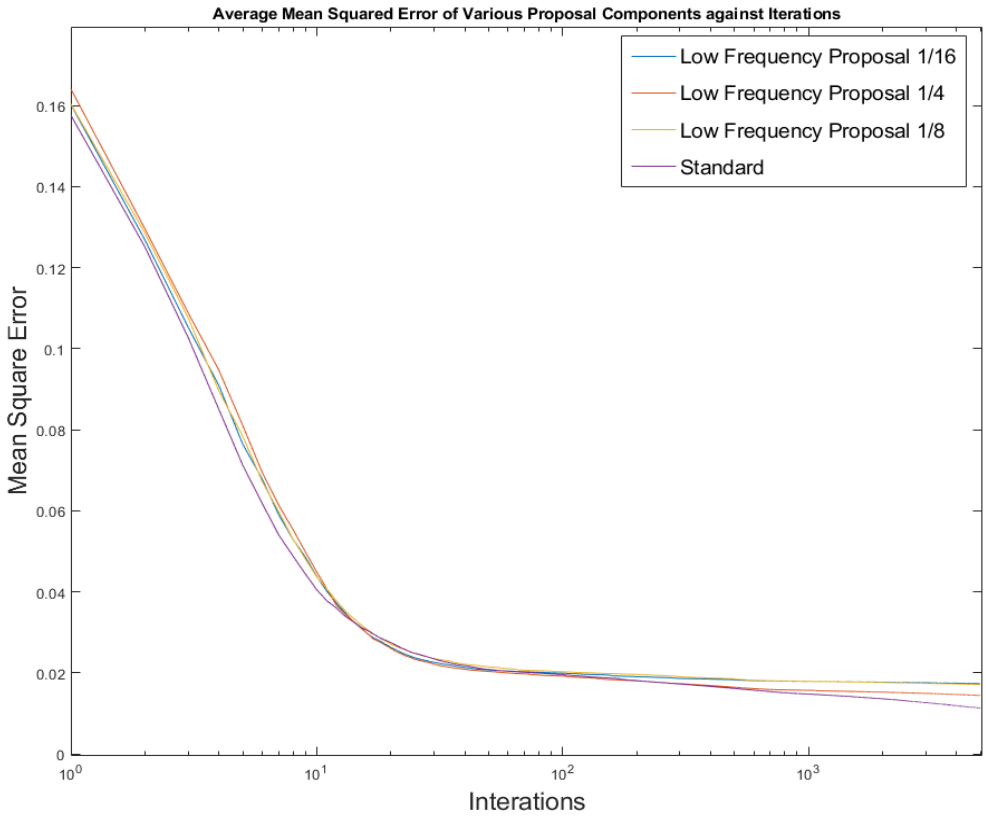


Figure 10

4.2.2.2 The Adaptive Proposal

The Adaptive Proposal did not produce a definitive improvement in results compared to the standard implementation. It produced an increase in the average error.

However, it did have a lower ratio of queries to iterations than the standard implementation. In addition to a greatly reduced average execution time.

The average query count to iteration ratio was for the standard methods was 1.0067, and average across the adaptive method was 0.9526. This is a system agnostic metric and shows that the adaptive proposal indeed results in less query. In addition to a 45% reduction in average run time for Adaptive Proposal 1 compared to standard. The run times are comparable because they were tested on the same system under the same load.

Test Configuration	*Average query count*	*Average Repeat Steps*	*Queries per iteration*	*Average run time*	*Average minimum MSE*	*Test System*
AP 1	3985.41	562.55	0.9572	77.20	1.809E-02	System 1
AP 2	4041.24	672.59	0.9504	88.36	1.309E-02	System 2
AP 3	3833.70	714.93	0.9503	87.35	1.380E-02	System 2
Standard	4609.19	n/a	1.0067	140.05	1.117E-02	System 1

Table 2 Adaptive Proposal Comparison to Standard

Neural Network	*GoogLeNet*	*InceptionV3*	*NASNetLarge*
System 1	*18.0868*	*28.2768*	*96.8329*
System 2	*14.2394*	*19.4142*	*100.0019*

Table 3 Seconds to classify an image 1000 times.

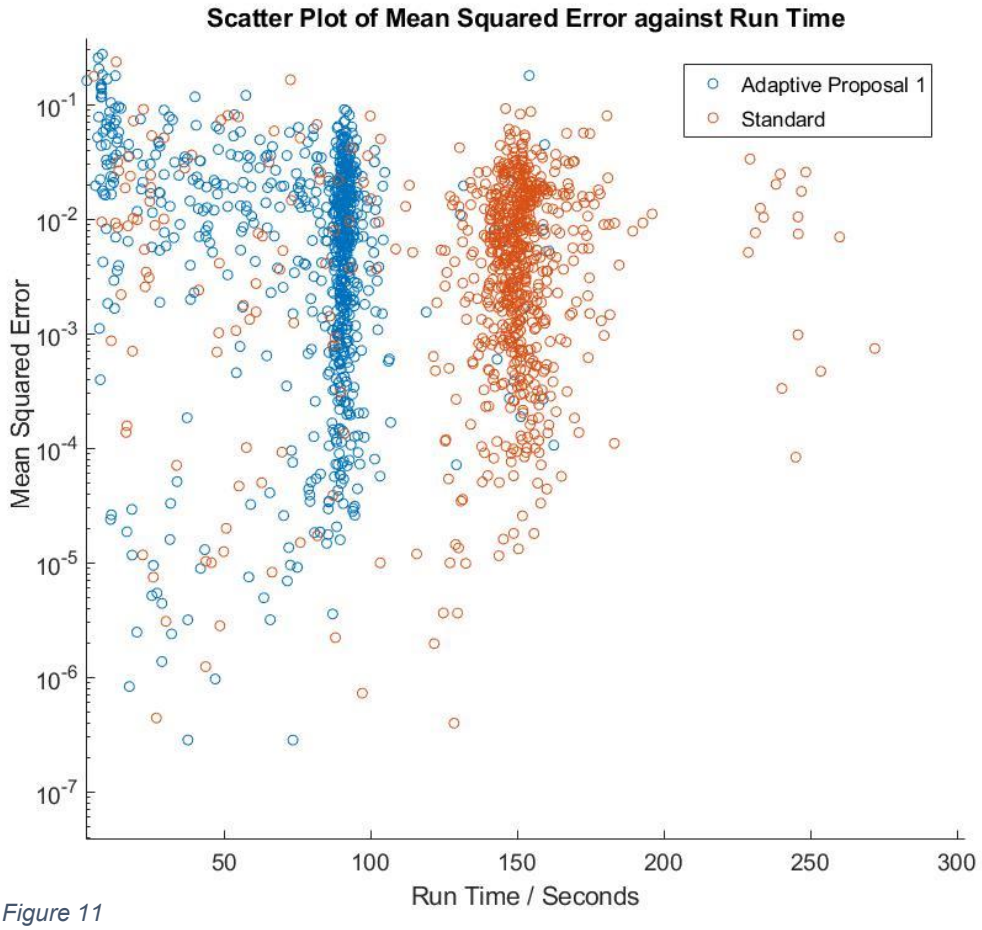


Figure 11

The 5.4% reduction in average query count is modest. Yet the run time decrease is not. As shown in figure 11. This indicates that the repeat step is saving a lot of computational time on not having to recalculate the orthogonal proposal in addition to not having to query the neural network quite as frequently. A better optimised back up proposal would likely reduce the scale of the run time difference.

My implementation of the Adaptive Proposal does consider cases beyond the first repeat differently. If a repeat step success is follow by a repeat step success the step length multiplier is raised to the power of current number of repeat steps to further increase the step length. Figure 12 shows that the performance of the Adaptive Proposal is poor in the early stages of the algorithm. This indicates that the repeat adjustment parameter is not optimal because the testing using a greater than 1 maximum repeat limit quickly catch up to standard BA. This is because there repeat adjustment parameter is square and cubed in some repeat steps increasing progress further. Yet the single max repeat step continues to lag behind in average MSE over its run.

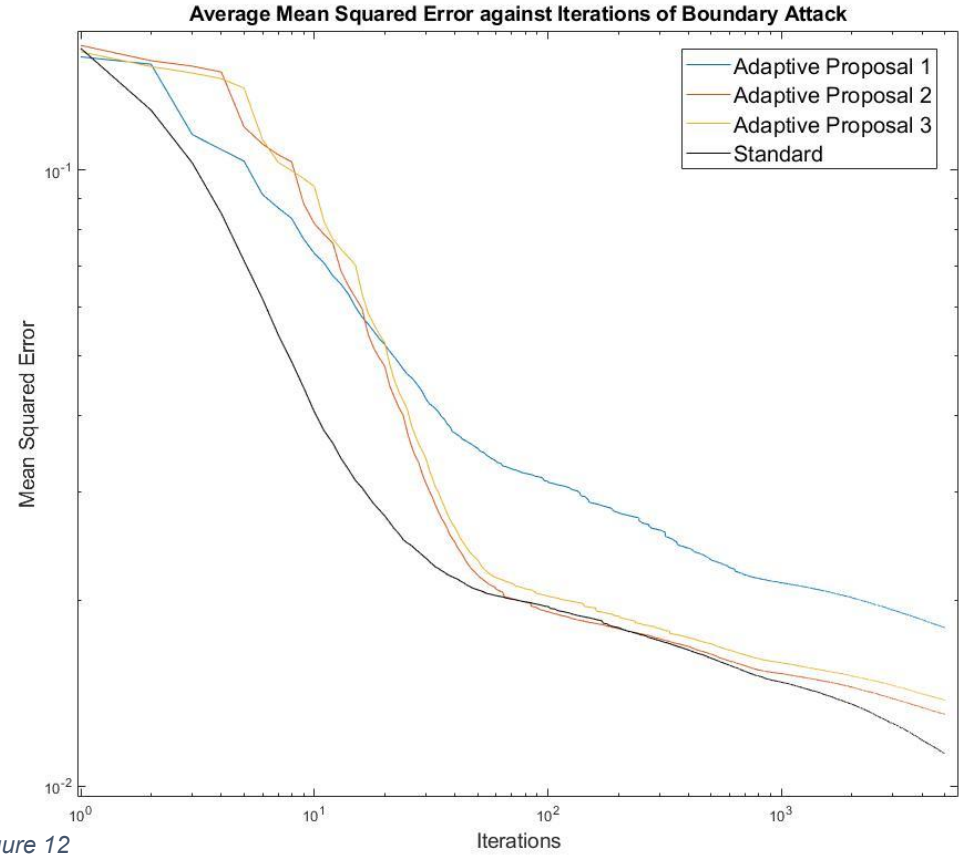

Average Mean Squared Error against Iterations of Boundary Attack
Adaptive Proposal 1
Adaptive Proposal 2
Adaptive Proposal 3
Standard
Mean Squared Error
Iterations
10^-1
10^-2
10^0
10^1
10^2
10^3

Figure 12

4.2.3 Boundary Attack Core Implementation.

I am confident in my BA core implementation as my program behaves as expected over a large number of iterations in comparison to (Brendel, Rauber and Bethge, 2018) work. I did not perform extensive comparable testing to their methods due to it being computationally prohibitive. I ran a single image test to 200,000 iterations and it behaved as their paper indicated it would.

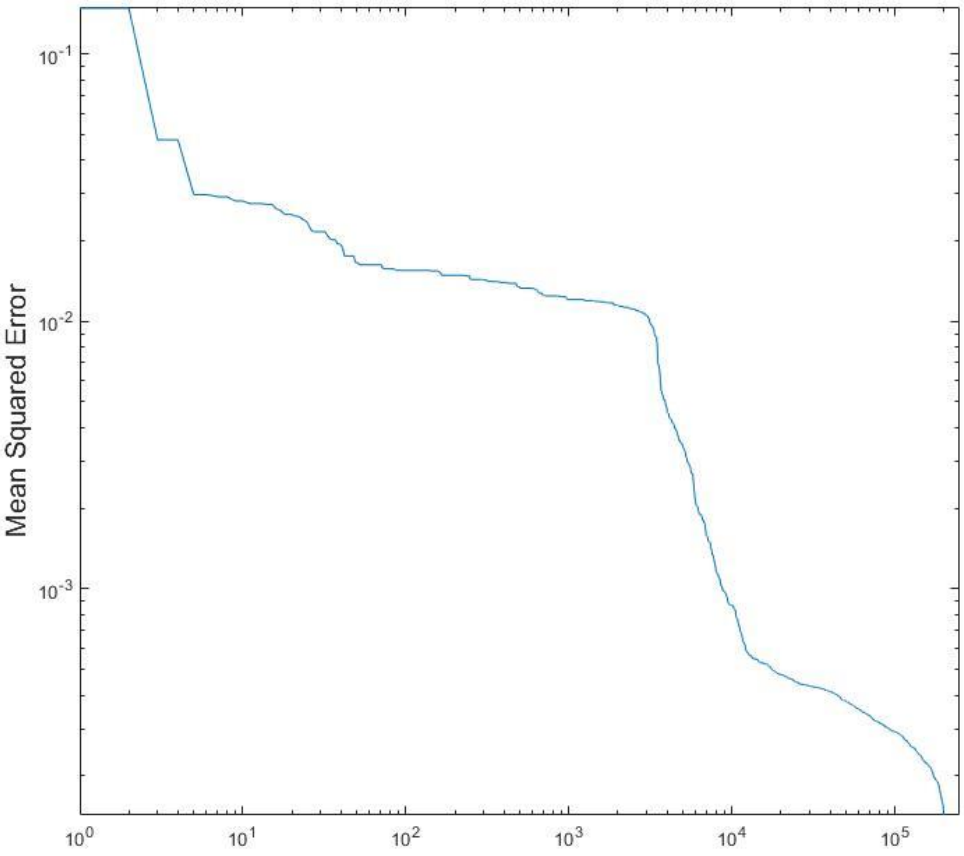


Figure 11 MSE of a single image to 200,000 iterations

4.2.4 Sanity Check

Boundary attack requires that the original image be correctly classified. This is recorded in the failed sanity check statistic. Every test had a 222 failed sanity checks. This is as expected because they were being run across the same images with the same neural network and were simply a validation statistic. The 1 failed image minimum is also explained by the label less class ‘crane’, the bird.

4.3 Results Images

4.3.1 Example Successful Images

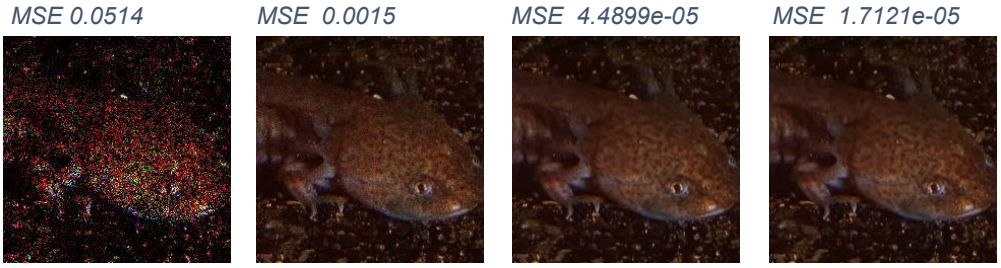


Figure 12 A successful run of Boundary Attack. True label 'axlotl' Predicted final label "platypus"

4.3.2 Example Failure images

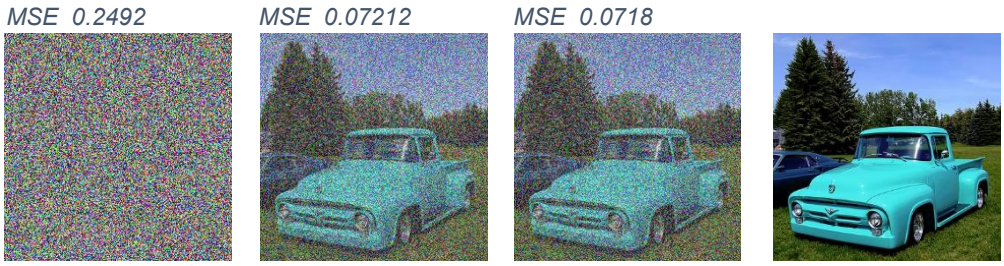


Figure 13 A failed run of Boundary Attack. True label 'pickup'. Predicted final label 'quilt No progress was made on the first iteration.'

4.3.3 Lowest Error Image

The lowest error adversarial image produced was in a run of the Low Frequency Initialiser with a DCT reduction of 1/8. This was simply a random, remarkably successful result. The final error of the image was 3.3629e-08 before the exit condition stopped the run as the error threshold limit was reached.

Figure 4 Classified as a 'pedestal' True label 'Military Uniform

Chapter 5: **Evaluation**

5.1 Project Evaluation

5.1.1 Strengths of the project

I implemented the whole of core BA. Implementation was hard, but the implementation of Boundary Attack was successful. I learnt a lot doing this and expected it to be hard. In balance, it was an invaluable personal learning experience. I chose a difficult project because I wanted to stretch myself. This meant the project was an extra-ordinary growth opportunity. It proved my ability to do self-motivated solo work and tackle tough tasks head on.

My secondary research was outstanding. I feel the first and second chapter demonstrate a deep level of understanding with the concepts at hand.

I was meticulous; my research effectively informed the design by identifying the areas that could be implemented as separate components. The design allowed for the flexible investigation of various methods and future expansion.

This meant I was able to implement a novel proposal optimization – which I termed the Adaptive Proposal. Having implemented BA and tested other methods I knew where potential inefficiencies lay.

At various points in the project, I had to be efficient with time and drop elements of the project. This meant I could explore more fruitful research areas. My time management proves a good level of efficiency and planning.

5.1.2 Weaknesses in the project

The lack of user testing is the primary weakness of the project. I did not get images of the required quality soon enough to perform the testing that I wanted to in time. Therefore, aims in the project that relied on user testing were shelved.

Even on the same image, BA has extremely variable results. This range can be so significant that it negates even large efficiency savings. The most rigorous way to entirely account for this is more extensive repeat testing than was feasible.

The exit condition was too stringent and limited the conclusions that could be drawn. Individual runs ended too quickly which meant results were more similar. Longer runs would show greater difference. The exit condition was stringent as I aimed to reduce the time to retest.

As shown in the various figures the initial stage is already reasonably fast as shown by the need to use a log scale to visualise a difference in performance. Alterations exclusively affecting the initial error are unlikely to net a significant improvement in the speed of the algorithm.

The results extraction script I created was more convoluted than ideal. As problems were found, I had to make changes to the data derivation script to compensate for errors which wasted time. These changes made the script far less usable by others as edits need to be made to the script to compensate for inconsistencies in different result sets. Testing should have been planned more with the initial design.

MATLAB led to a few inherent weaknesses of the project, but after some cost-benefit analysis I think it was for the best. I chose to use MATLAB, which gave significant benefits to plotting data, inspecting variables and rapid prototyping. Rapid prototypes potentially led to more mistakes. Using MATLAB meant I had to reimplement the entire of Boundary Attack, which was both a benefit to the project and a massive drain on time. If I had used Python I imagine by the end of the project I would have wished I was using MATLAB and visa versa.

5.2 Legal, Ethics and Sustainability

5.2.1 Image Use legality

I mostly used the ImageNet dataset. ImageNet does not own the copyright to the images they licence the ability to download these images for non-commercial research and education. Details of the licence are found on the ImageNet website. As of 12/04/2020 ImageNet's download services are down. However, they are providing a mirror on Kaggle of a portion of their dataset.

The remaining images I used were photos I had taken, or access was provided by the image owner.

5.2.2 The Legal Considerations of interfering with Object Detection Systems.

The key legal consideration would be to not interfere with the object detection systems of others without permission and in a safe setting. Interfering with systems without permission would likely fall foul of the Computer Misuse Act 1990 Section 3, which prohibits unauthorised acts with the intent to impair the operations of a computer. Computer in this context covers any particular computer, program, or data. Object detection systems would fall under this in my assessment. Particularly when they are part of a larger system like a car. (Computer Misuse Act 1990) I stayed well away from this problem by developing and testing in a closed system.

5.2.3 Is it ethical to research improvements to algorithms designed to trick safety systems?

It is valuable to perform this research so that developers are aware of the flaws and problems with object detection systems so the durability of these systems can be pushed, reducing the risk of fault.

If this research is not done there is increased risk malicious actors will identify faults with these systems and be able to exploit them, which would put users of these systems at increased risk.

Furthermore, it is important to advance the robustness of an object detection system now before they are used widely in uncontrolled high-risk settings like automated vehicles.

It would be best to have a culture of robustness and safety ingrained in automation going forward as these systems would otherwise be extremely dangerous.

At the moment, this is in contrast to a space like data security where vulnerability testing often needs to be performed on live systems, with real user data. It is clear that while this sort of security testing needs to be done, it would be preferable to do testing in a closed system while it is still possible.

This is a risk management problem with the risk increased if faults are not published and managed. There is a clear parallel with benevolent penetration testing. However, without the associated difficulties with safely disclosing vulnerability information that exists in security research. In this space, we are still able to share vulnerabilities with far less risk as mass deployment has yet to happen.

In conclusion, it would be unethical not to perform research in this space if there continues to be a push towards automation in uncontrolled settings. This sort of testing should help increase safety standards within systems using object detection neural networks. (Wong. K, 2020)

5.2.4 How does sustainability relate to this project?

This is at its core a research project into increasing computational efficiency of an algorithm. Therefore, as a side product of this goal sustainability is increased. However, computational sustainability is a minor consideration in this project as this project aims to push the safety of object recognition systems rather than to be used long term. Therefore, it will have a fractional computational footprint compared to that of a widely used networking protocol, for example. These larger-scale more resource-hungry systems are a place where sustainability should be more of a focus. In addition, as this project aims to push for greater robustness and reduced risk, sustainability is further reduced in importance in this project. (Milano, 2014)

5.3 Factors Impacting the Project

5.3.1 COVID-19 and the effect on the project

Due to the unprecedented disruption of the COVID-19 pandemic, aspects of this project were both cut short and forced to compete with other newly assigned coursework. The project deadlines were moved far less than other coursework due to its proximity to exams. This resulted in an overlap of coursework and finishing this project. This reduced the priority I was able to give to the project in the final five weeks before it was due. Devoting as much time to my project as initially planned would cut into vital time revising for imminent exams and completing exam replacement coursework.

Chapter 6: **Conclusion**

6.1 Summary of Results

6.1.1 The methods worked

The Informed method worked as intended. It greatly reduced the initial error while being simple and quick to compute. It did introduce a failure rate that did not previously exist, but the rate was low. Furthermore, as shown by the reduction in average final error, the images that failed were not going to be successful over a small number of iterations.

The Low Frequency Initialiser was created as a validation method, yet it reduced the initial error without any introduced failure rate. For natural images, this proved to be an objective improvement over the standard initialisation.

The Adaptive Method showed a great deal of promise but it is difficult to draw very concrete results. It was shown to have a lower average runtime and queried the neural network 5% less frequently per iteration than the standard Boundary Attack. However, the average final error was higher than standard Boundary Attack. This merits further testing because it achieved this increased performance with minimal optimisation.

6.1.2 These methods did not work

I expected the Negative Informed Method to reduce initial error. However, I over-estimated how much the most significant bit dictated the initial error. It did not produce an initial error any different than sampling from a uniform distribution.

The reimplementation of the Low-Frequency Proposal was an unexpected failure, and it is clear that more time was needed to identify the fault.

6.1.3 Did I achieve my aims and objectives?

I implemented Boundary Attack in MATLAB with a good degree of success. I succeeded in implementing standard Boundary Attack as shown by the long single image test.

I implemented an alternative improved proposal distribution for the Boundary Attack method in the form of the Adaptive Proposal. Its results were promising but not conclusive. Implementing this idea was a great success, given that the core aim of my project was to try and implement a novel improvement to the Boundary Attack method. I was able to conceive of and implement this method due to a deep understanding of Boundary Attack.

I implemented a heuristic initialisation method. This method introduced a failure rate of around 10%. However, it reduced the initial error by 400%. I consider this quite successful.

My investigations into hyperparameter adjuster improvements were minimal due to more promising improvements in other areas. Hyperparameter adjuster improvement proposals are discussed below in 6.2.2.

The altered stop function worked to curtail runs aggressively in an attempt to accelerate testing. However, it required user testing to be useful.

The most significant detriment to my aims and objectives was limited time for testing. The parameters I chose for the exit condition required adjustment, but the testing in order to implement this would cut into time testing images themselves. This was also true for the Adaptive Proposal which needed adjustments made to the step length multiplier only possible with further testing.

6.2 Future Work

6.2.1 Boundary Attack as a Method of Adversarial Image Generation

The Boundary Attack method is exceptionally slow. This is a perennial feature of black-box attacks as querying networks itself is a slow process. Also, they become increasingly slow as neural networks become more complex with increasing layer counts. Improvements can be made yet will never be as fast as white-box methods. Methods developed in this project provide a possible route to improvements in speed.

6.2.2 Directions to explore further

The non-batched based hyperparameter adjustment showed promise and merits investigations in the future. It would require methods to prevent run away hyperparameters like capping the values or resetting them periodically. I see benefits here in finding more optimal or faster solutions.

Also, there is room to optimise hyperparameter adjustment through early batch completion.

A more complex heuristic could be created that would further improve the initial image. The Positive Informed method shows that a quick to compute heuristic can reduce the initial error by a large amount. A linked challenge would be reducing the failure rate that this method would introduce.

Due to time constraints, the components and systems developed were tested on a less up-to-date neural network. Any future project would have to retest these new methods on a more modern system.

The Adaptive Proposal needs further refinement, optimisation, and investigation. It showed a slightly reduced average query count per iteration and a large reduction in average run time, but its average final mean squared error was worse than standard Boundary Attack.

A hybrid development approach would have benefited this project. Adapting the code given in the original implementation to my modular design would gain the benefits of both. It would have allowed more potential improvements to be implemented and have resulted in a stronger project.

References

1. Goodfellow, I., Papernot, N., Huang, S., Duan, R., Abbeel, P. and Clark, J. (2019). *Attacking Machine Learning with Adversarial Examples*. [online] OpenAI. Available at: https://openai.com/blog/adversarial-example-research/ [Accessed 7 Nov. 2019]

2. Goodfellow, I., Shlens, J. and Szegedy, C. (2015). *Explaining and Harnessing Adversarial Example*. 1st ed. International Conference on Learning Representations.

3. Guo, C., Frank, J. and Weinberger, K. (2019). *Low Frequency Adversarial Perturbation*. [pdf] Available at: https://arxiv.org/abs/1809.08758.

4. Brendel, W., Rauber, J. and Bethge, M. (2018). *Decision-Based Adversarial Attacks: Reliable Attacks Against Black-Box Machine Learning Models*. International Conference on Learning Representations.

5. LeCun, Y., Bengio, Y. and Hinton, G. (2015). Deep learning. *Nature*, 521(7553), pp.436-444.

6. Tramer, F., Kurakin, A., Papernot, N., Goodfellow, I., Boneh, D. and McDaniel, P. (2018). Ensemble Adversarial Training: Attacks and Defenses. International Conference on Learning Representations.

7. Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z. and Swami, A. (2017). *Practical Black-Box Attacks against Machine Learning*.

8. Gonzalez, R. and Woods, R. (2017). *Digital image processing*. 4th ed. Pearson, p.132.

9. Papernot, N., McDaniel, P., Jha, S. and Swami, A. (2016). *Distillation as a Defense to Adversarial Perturbations against Deep Neural Networks*. 2nd ed.

10. Moosavi-Dezfooli, S., Fawzi, A., Fawzi, O. and Frossard, P. (2017). *Universal Adversarial Perturbations*. 1st ed.

11. Moosavi-Dezfooli, S., Fawzi, A. and Frossard, P. (2015). *DeepFool: a simple and accurate method to fool deep neural networks*. [online] arXiv.org. Available at: https://arxiv.org/abs/1511.04599 [Accessed 15 Oct. 2019].

12. Tesla.com. (2019). *A Tragic Loss*. [online] Available at: https://www.tesla.com/en_GB/blog/tragic-loss [Accessed 28 Nov. 2019].

13. *Computer Misuse Act 1990 Section 3,* Available at: http://www.legislation.gov.uk/ukpga/1990/18/section/3/2015-05-03 [Accessed 15 Apr. 2020]

14. Wong, K. *The development of computer ethics: Contributions from business ethics and medical ethics.* SCI ENG ETHICS 6, 245–253 (2000). https://doi-org.ezproxy.library.qmul.ac.uk/10.1007/s11948-000-0052-9

15. Milano, M., 2014. *Where Computer Science meets Sustainable Development.* IEEE Transactions on Computers, 63(1), pp.88-89.

16. Povolny, S. and Trivedi, S., 2020. *Model Hacking ADAS To Pave Safer Roads For Autonomous Vehicles* | *Mcafee Blogs*. [online] McAfee Blogs. Available at: <https://www.mcafee.com/blogs/other-blogs/mcafee-labs/model-hacking-adas-to-pave-safer-roads-for-autonomous-vehicles/> [Accessed 2 May 2020].

Appendix A – Abbreviations and Glossary

6.3 Abbreviations

ABA – Adaptive Boundary Attack

BA – Boundary Attack

CNN – Convolutional Neural Network

DNN – Deep Neural Network

EAT – Ensemble Adversarial Training

FGSM – Fast Gradient Sign Method

IM – Informed Method

IDCT – Inverse Discrete Cosine Transform

DCT – Discrete Cosine Transform

NI – Negative Informed (Method)

PI – Positive Informed (Method)

PTA – Papernot Transfer Attack

UDI – Uniform Distribution Initialiser

6.4 Glossary

Adversarial Criterion – The method by which an image is identified to be adversarial. Misclassification, Targeted Misclassification and Top-k Misclassification are examples.

Class – The label identifying the content of the image.

Misclassification – Whether the model assigns a class to an image other than the class that it is correctly labelled as.

Targeted Misclassification – Whether the model assigns a chosen class to an image rather than the class it is correctly labelled as.

Top-k misclassification – Whether model assigns K different classes above that of the correctly labelled class of the image.

Confidence score – the percentage score a classification model gives an image for the chosen label it has given an image. High confidence scores are assumed to be more likely to be correctly classified.

Appendix B – User Manual

Initialising Boundary Attack

Boundary Attack Class and the Adaptive Boundary Attack Class both need to be initialised for each attempt with 11 parameters.

1. An initial image generator, Inheriting from Initialiser_Interface.m
   There are 4 suitable classes. Informed_Initializer.m , Informed_Low_Initializer.m , Uniform_Distribution_Initilizer.m and Low_Frequency_Initializer.m

   a. Uniform_Distribution_Initilizer.m requires no parameters.

   b. Informed_Initializer.m requires one parameter, bit depth.
      i. bit depth – Integers. Non 0 in the range -7 to 7. Defines the number of bits to be removed by the Initialiser from the original image to create the initial image. A negative value removes the most significant bits from the image. Positive the least significant bits.

   c. Low_Frequency_Initializer.m requires one parameter, dct reduction.
      i. dct reduction – Double, defining the proportion of the discrete cosine transform matrix zeroed in the production of low frequency noise. the Smaller the value the lower the frequencies maintained. Values between 0 – 1.

   d. Informed_Low_Initializer.m requires two parameters, bit depth and dct reduction.
      i. bit depth – described above.
      ii. dct reduction – described above.

2. A proposal distribution, Inheriting from the Proposal_Distribution.m

   a. There are 2 classes suitable for Boundary Attack and 1 suitable class for Adaptive Boundary Attack. Adaptive Boundary Attack can only use Adaptive_Proposal.m and Boundary Attack with either the Low_Frequency_Proposal.m or the Gaussian_Proposal.m.
      i. Adaptive_Proposal requires 3 parameters.
         1. max repeat steps – caps the number of times a successful orthogonal image can be reused.
         2. backup proposal – a proposal distribution to be used in non-repeat steps.
         3. step length multiplier – used to reduce or increase the original step size used in repeat steps.
      ii. Gaussian_Proposal has no parameters.
      iii. Low_Frequency_Proposal has one parameter, dct reduction
         1. dct reduction – described above.

3. An adversarial criterion, Inheriting form Adversarial_Criterion.m

    a. There is one supported criterion, Misclassified.m. This criterion must be initialised with a categorical array of labels for the class of the current target image.

4. A hyperparameter adjuster, Inheriting from Hyperparameter_Adjuster.m

    a. Only supported adjuster is Trust_Region.m. Trust_Region must be initialised with 7 parameters.
        i. Upper bound step in – the upper bound marking too much in step success. If the probability of a success in a batch is above this the in step size is multiplied by the adjustment rate.
        ii. Lower bound step in – the low bound marking too little in step success. If the probability of a success in a batch is below this the in step size is divided by the adjustment rate.
        iii. Upper bound orthogonal step in – the upper bound marking too much orthogonal step success. If the probability of a success in a batch is above this the orthogonal step size is multiplied by the adjustment rate.
        iv. Lower bound orthogonal step in – the upper bound marking too little orthogonal step success. If the probability of a success in a batch is below this the orthogonal step size is divided by the adjustment rate.
        v. Orthogonal Step in batch size – the number of orthogonal results before the orthogonal step size is evaluated for adjustment
        vi. Step in batch size – the number of in step results before the in step size is evaluated for adjustment.
        vii. Adjustment rate – the multiplying or dividing rate when adjusting the hyperparameters up and down. Must be greater than one.

5. A neural network, Inheriting from Neural_Network.m
    a. There are 3 implemented that can be used. InceptionResNetV2, InceptionV3 and GoogLeNet. These class do not require any additional parameters.

6. An Exit_Condition, Inheriting from Exit_Condition.m.
    a. The only supported exit condition is MS_Threshold.m.
        i. MS_Threshold must be initialised with 3 parameters. The MSE Threshold in type double, the max number of iterations of Boundary Attack and the maximum number of iterations without change. Defaulted to 0.0000002, 5000, 400 respectfully.

7. A Target image. The image that you wish to create an adversarial image of.
    a. This image must be single or three channel 8 bit and preferably with dimension greater than the input size of the Neural Network or it will be scaled up.

8. A 1D array of true labels. Must be of type categorical with the first value in the array not containing a label and from there a continues list of labels.

9. The last three parameter control the output of the results images.
    a. x and y are used to label the folder of individual results images. x in my testing was the number of passes over the list of classes and y was the current class.
    b. The test category is a used to label the folder that output individual runs folders are saved under. In addition, the name is appended to the results output into the working directory by BA_Test.m

Running Boundary Attack

With Boundary Attack or Adaptive Boundary Attack Initialized. The processing of an image can be started by calling Calculate_Adversarial_Image() on the Boundary Attack or Adaptive Boundary Attack object.

Various statistical results are saved into cell arrays for each version of Boundary Attack run. Each cell containing the results for an image that was processed.

See main.m and test.m for the results testing script.

BA_Test.m can be used for testing with the parameter classes altered as desired for each run.

Adaptive_BA_Test.m is the variant class for testing Adaptive Boundary Attack.

BA_Test and ABA_Test require a path to the ImageNet data set with each class contained in separate files as they come in the ImageNet data set available at https://www.kaggle.com/c/imagenet-object-localization-challenge

Appendix C – System Manual

MATLAB Version R2019B was used exclusively for this project.

The MATLAB addons used were:
Computer Vision Toolbox Version 9.1
Deep Learning Toolbox Version 13.0
Deep learning Toolbox Model for GoogLeNet Network Version 19.2.0
Deep Learning Toolbox Model for Inception-v3 Network Version 19.2.0
Deep Learning Toolbox Model for Inception-ResNet-v2 Network Version 19.2.0
Deep Learning Toolbox Model for NASNet-Large Network Version 19.2.0
GPU Coder Version 1.4
GPU Coder Interface for Deep Learning Libraries Version 19.2.2
Image Processing Toolbox Version 11.0
MATLAB Coder Version 4.3
MATLAB Support for MinGW-w64 C/C++ Compiler Version 19.2.0

All testing and development was performed on Windows 10 computers with Nvidia GPUs and Intel CPUs

System 1 - Intel I7-7700K with 2 Nvidia GTX 980

System 2 - Intel I7-9750H with Nvidia RTX 2060

Compatibility with other operating systems and versions of MATLAB hasn't been tested and wasn't planned for.

Project Source File List with Descriptions

This files listed below were all written by me. Some are reimplementation in MATLAB of existing methods as described in the report.

Adaptive_BA_Test.m – The testing script that will run batches of Adaptive Boundary Attack.

Adaptive_Proposal.m – My implementation of the Adaptive proposal method the I developed as part of this project. It is a inherits from handle and Proposal_Distribution.

Adaptive_Boundary_Attack.m – The versions of Boundary Attack Core to be used when using the adaptive proposal due to limitations in the original Boundary Attack class design.

Adversarial_Criterion.m – Interface for adversarial criterions.

BA_Test.m – The testing script that will run batches of Boundary Attack with various different components and parameters.

Boundary_Attack.m – The core of the Boundary Attack algorithm. Controlling the logic of the system and managing the calls to components. In addition to recording data about each run.

Derived_Data.m – Script to take the raw data from a run over of BA_Test.m or Adaptive_BA_Test.m and derive more data.

Exit_Condition.m – the exit condition interface.

folder_list.mat – Saved output of the dir() function. Used to making a list of each folder that contains training data.

folder_list_laptop.mat – Saved output of the dir() function. Used to making a list of each folder that contains training data. For the training data location on my second test machine.

Gaussian_Proposal.m – My reimplementation of the standard proposal distribution for Boundary Attack. Inherits from Proposal_Distribution

GoogLeNet.m – Class for querying the GoogLeNet neural network. Inherits from the Neural Network interface

Hyperparameter_Adjuster.m – the hyperparameter adjuster interface

InceptionResNetV2.m – Class for querying the InceptionResNetV2 neural network. Inherits from the Neural Network interface

InceptionV3.m – Class for querying the InceptionV3 neural network. Inherits from the Neural Network interface

Informed_Initializer.m – class implementing the informed heuristic to generate initial images. It Inherits from Initialiser_Interface.m

Informed_Low_Initializer.m – A variant of Informed_Initializer where emptied bits are filled with low frequency noise rather than noise sampled from a uniform distribution.

Initialiser_Interface.m – Interface for initial image generations classes.

Label_Keys.mat – table containing the labels for a given folder in the imagenet training set. Found at https://www.kaggle.com/c/imagenet-object-localization-challenge Normalised as explained in dataset challenges.

Low_Frequency_Initializer.m – An initial image generator using low frequency noise using the IDCT reduction method.

Low_Frequency_Proposal.m – A reimplementation of the low frequency proposal in reference 3. Using the IDCT reduction method of generating low frequency noise. Inherits from Proposal_Distribution

Misclassifed.m – class for the standard misclassification adversarial criterion. Inherits from the adversarial criterion interface.

MS_Threshold.m – the modified end condition. Inherits from Exit_Condition.m Interface.

NASNetLarge.m – Class for querying the NASNetLarge neural network. Inherits from the Neural Network interface

Neural_Network.m – Interface to allow various classification system.

Proposal_Distribution.m – Interface for proposal distributions.

Trust_Region_Adjuster.m – the trust region inspired adjuster. Inheriting from Hyperparameter_Adjuster.m

Uniform_Distribution_Initilizer.m – For initialising Boundary Attack with an image sampled from a uniforme distribution. Inherits from Initialiser interface.

As seen, there is extensive sub classing and interfaces to allow for a flexible swapping of components to allow for expansion of scope in the project effectively.

Appendix D – Design Document

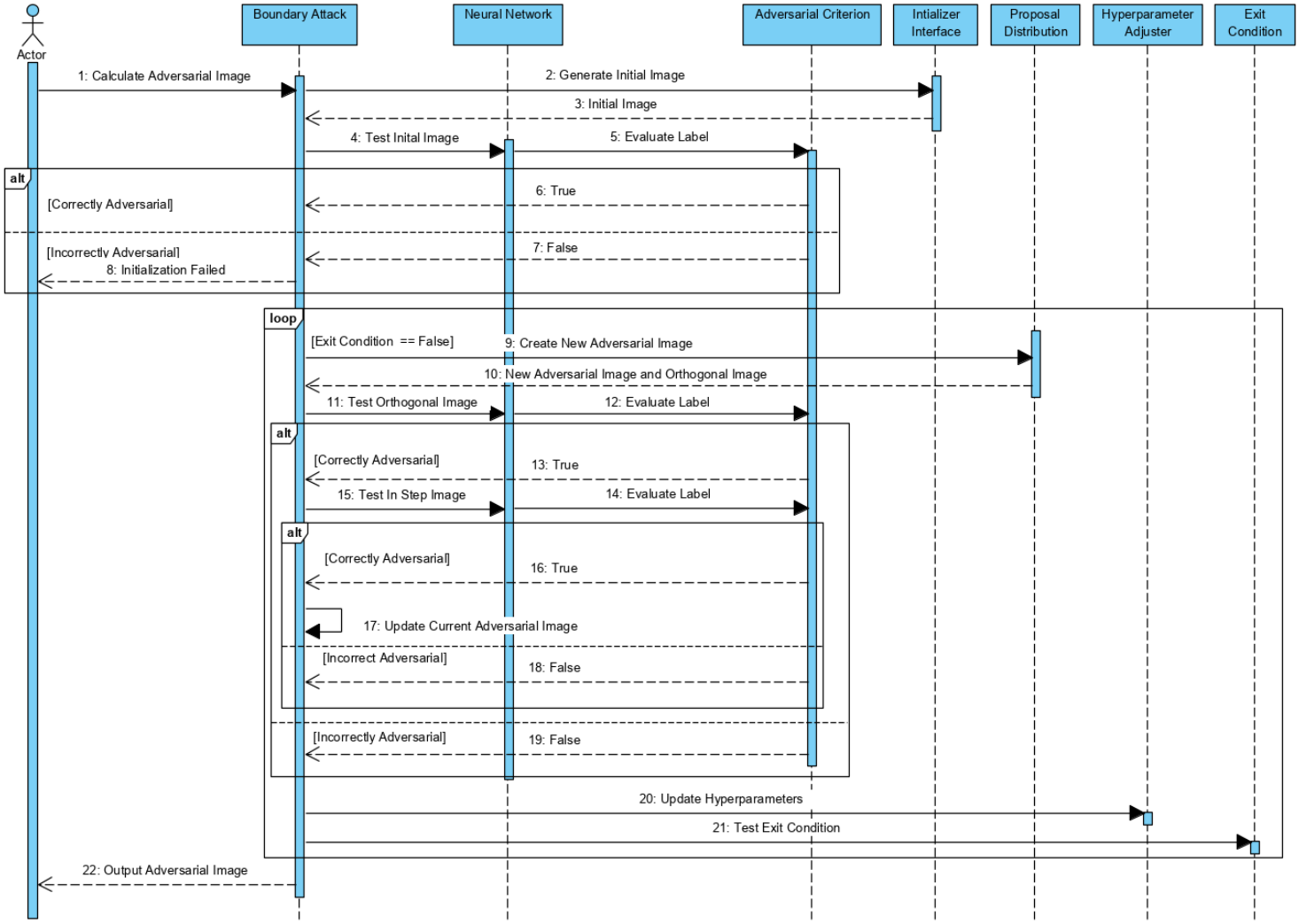
Visual Paradigm 16.0 was used for the creation of the class and sequence diagrams for this project.

I made the decision to aggressively decouple the program. I developed into separate classes so give myself flexibility to explore alternative methods. This is shown in the extensive use of interfaces to allow various component substitutions to be made to the core of the BA algorithm.

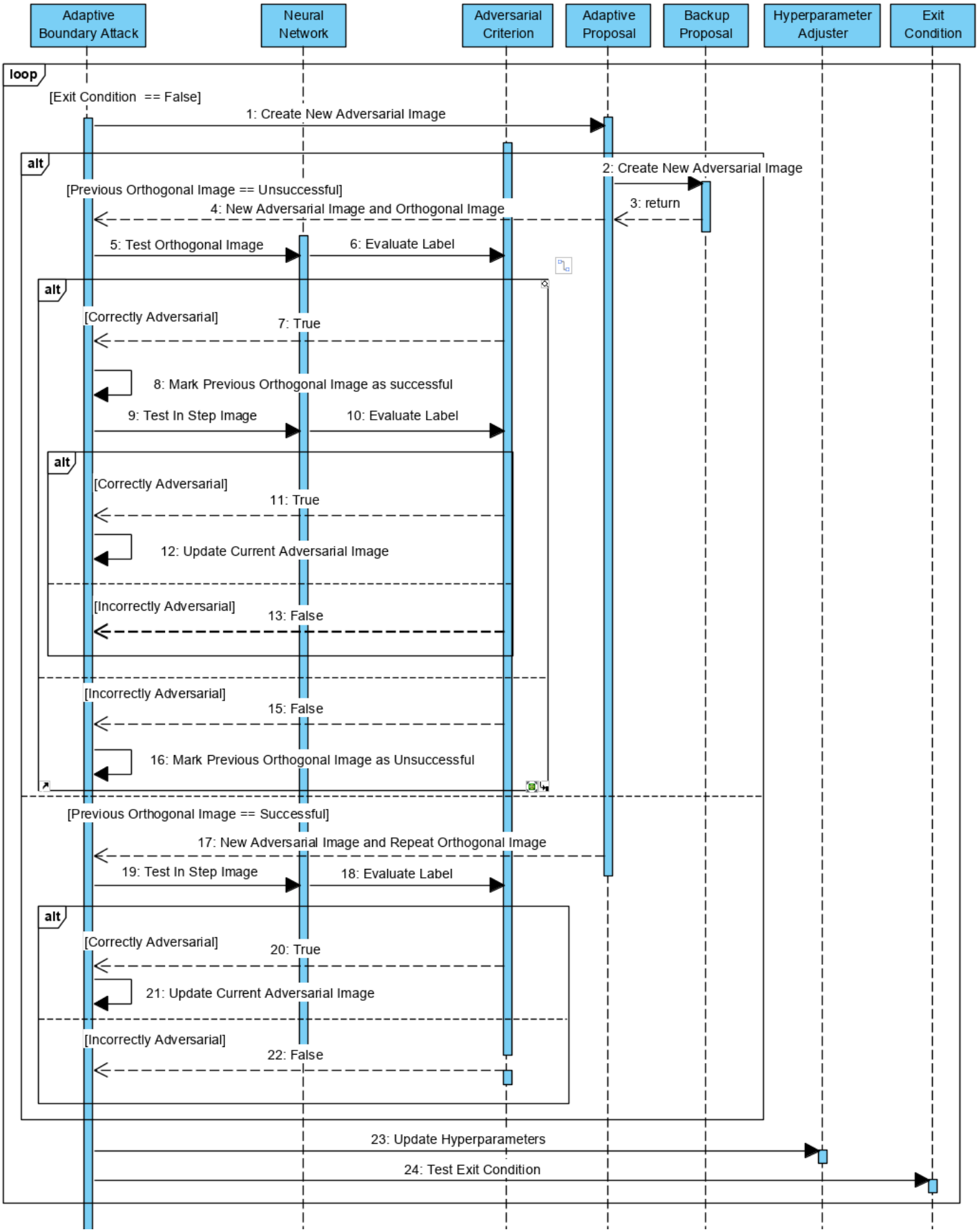
My original design did not allow for flexibility within the boundary attack class. Hence the need for the Adaptive Boundary Attack class. This could have been solved in design if I had considered that the improvements might require changes to the Boundary Attack core.

The design required that components be made as independently were possible. This was not always possible for the adaptive proposal changes needed to be made to the Boundary Attack Class hence the second adaptive boundary attack class. Due to early decisions that I could not change without an extensive rewrite.

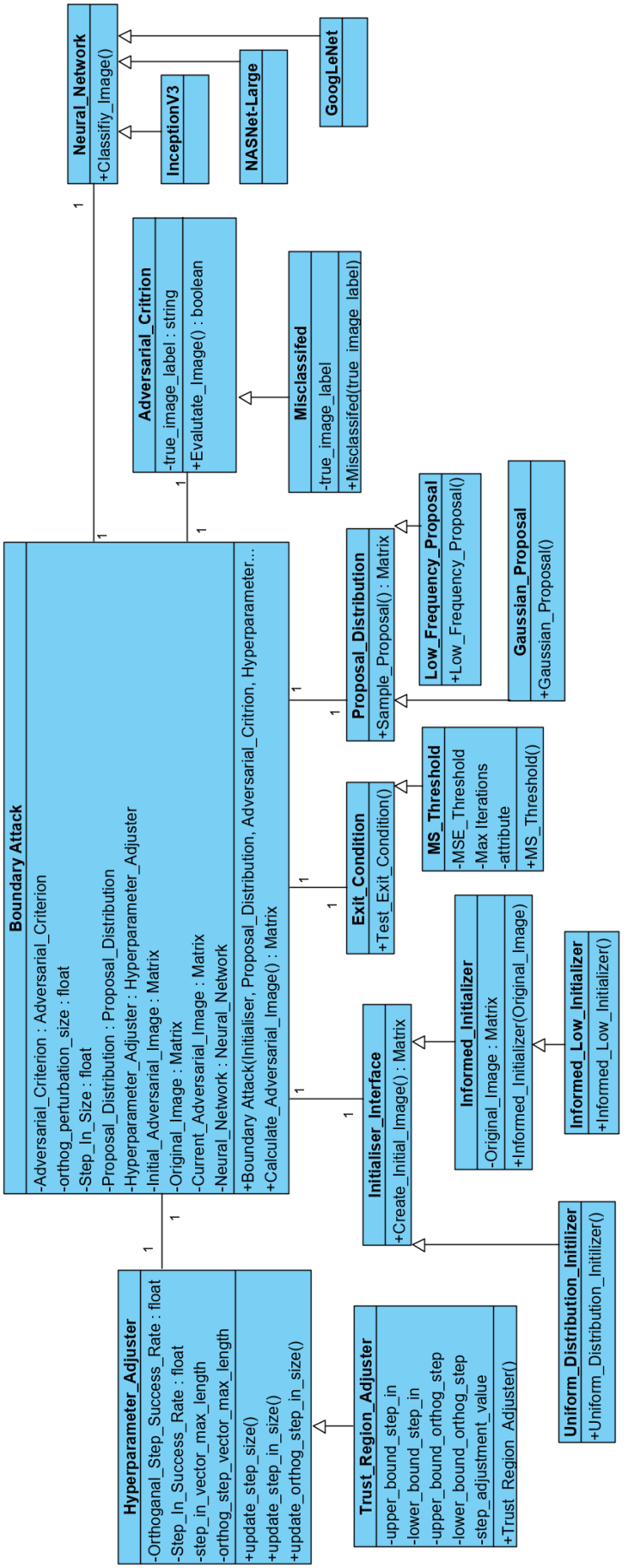
Boundary Attack Sequence Diagram



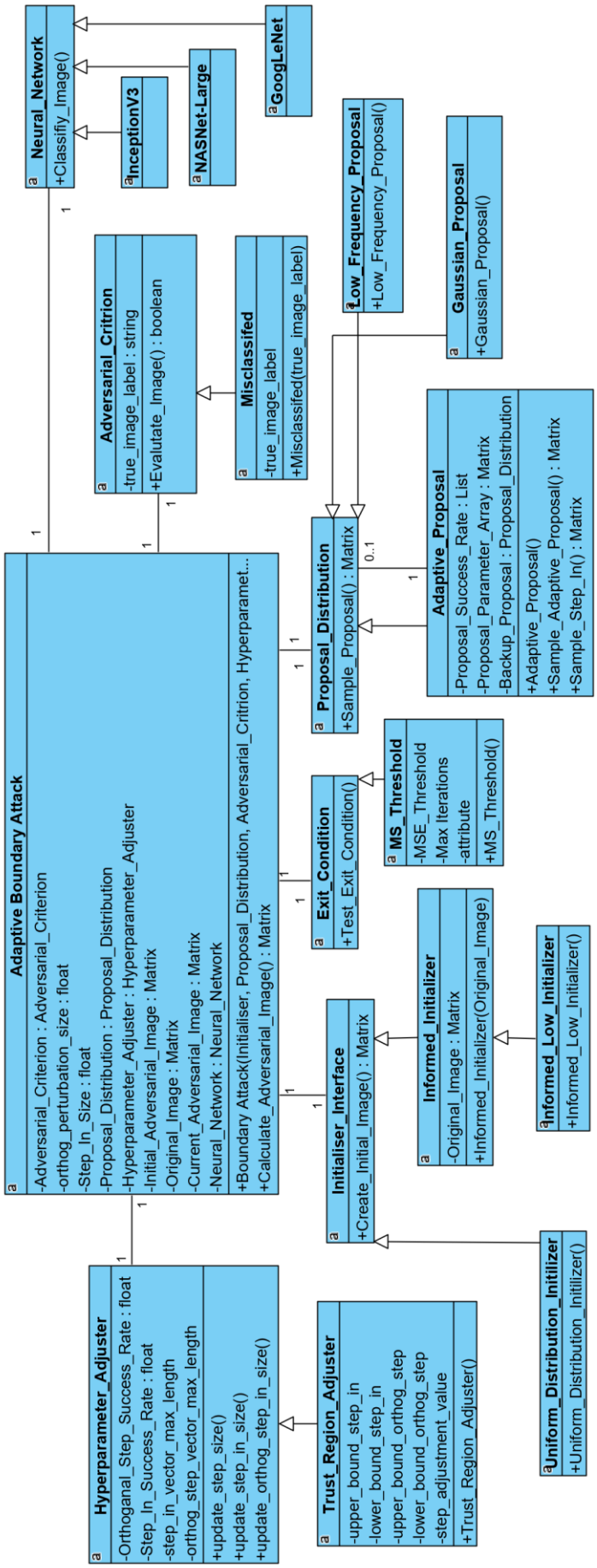
Adaptive Boundary Attack Iteration Sequence



The Boundary Attack Class diagram



Adaptive Boundary Attack Class diagram



Appendix E – Experimental Results

Output images and further results that would be impractical to display here are available at this link. Including the spreadsheet of results seen on the next page.

https://qmulprod-my.sharepoint.com/:f:/g/personal/ec15066_qmul_ac_uk/EpY_E-uKYvlMnYK67B-4vsUBGPiYQPAiqvQsjM4spY6E-A?e=cSFOPL

Column1	Test Name	Lowest_err	lowest_run_time	highest_run_time	failed_Initialization	failed_sanity_check	avg_run_time	avg_query_count	avg_intial_error	avg_minimum_e	repeat_steps	avg_iterations	query per iteration
System 1	Adaptive Proposal (1, Gaussian, 1.5)	2.82E-07	3.26	162.52	1	222	77.20	3985.41	0.15272	1.809E-02	548.91	4163.81	0.9572
System 2	Adaptive Proposal (2, Gaussian, 1.5)	2.99E-07	4.37	1.27E+02	1	222	88.36	4041.24	0.15938	1.309E-02	645.72	4252.29	0.9504
System 2	Adaptive Proposal (3, Gaussian, 1.5)	4.93E-08	8.56	1.16E+02	1	222	87.35	3833.70	0.15566	1.380E-02	714.93	4034.32	0.9503
System 2	Informed -1	3.21E-07	5.80	169.29	1	222	125.11	4363.89	0.16438	2.120E-02	n/a	4328.70	1.0081
System 2	Informed -2	3.55E-07	5.17	274.41	1	222	143.15	4519.72	0.15588	1.216E-02	n/a	4476.02	1.0098
System 2	Informed 5	1.52E-07	5.88	188.98	602	222	128.68	3718.27	0.01540	2.402E-03	n/a	3959.14	0.9392
System 2	Informed 6	1.95E-07	7.02	196.34	323	222	135.50	4080.61	0.01742	3.222E-03	n/a	4131.63	0.9877
System 2	Informed 7	3.46E-07	7.00	158.52	76	222	127.99	4450.12	0.04821	7.197E-03	n/a	4468.09	0.9960
System 2	Informed Low 5	5.02E-07	7.32	180.96	627	222	108.31	3229.71	0.01899	8.103E-04	n/a	3283.07	0.9837
System 2	Informed Low 6	3.96E-08	12.40	340.42	376	222	136.46	3718.69	0.01449	2.607E-03	n/a	3768.21	0.9869
System 2	Informed Low 7	2.72E-07	5.93	239.28	75	222	115.56	3942.95	0.03170	7.097E-03	n/a	3961.11	0.9954
System 1	Low Init 1/8	3.36E-08	6.09	268.45	1	222	159.60	4429.44	0.10169	9.937E-03	n/a	4403.03	1.0060
System 1	Low Proposal 1/16	2.46E-06	9.12	297.40	1	222	140.34	3710.11	0.16028	1.733E-02	n/a	4583.49	1.0769
System 1	Low Proposal 1/4	1.22E-06	8.28	221.25	1	222	182.83	4935.96	0.16413	1.442E-02	n/a	4463.72	1.0287
System 1	Low Proposal 1/8	6.94E-07	11.04	238.07	1	222	148.89	4591.68	0.16040	1.710E-02	n/a	4049.47	0.9162
System 1	Standard	3.96E-07	5.71	271.90	1	222	140.05	4609.19	0.15765	1.117E-02	n/a	4577.34	1.0070